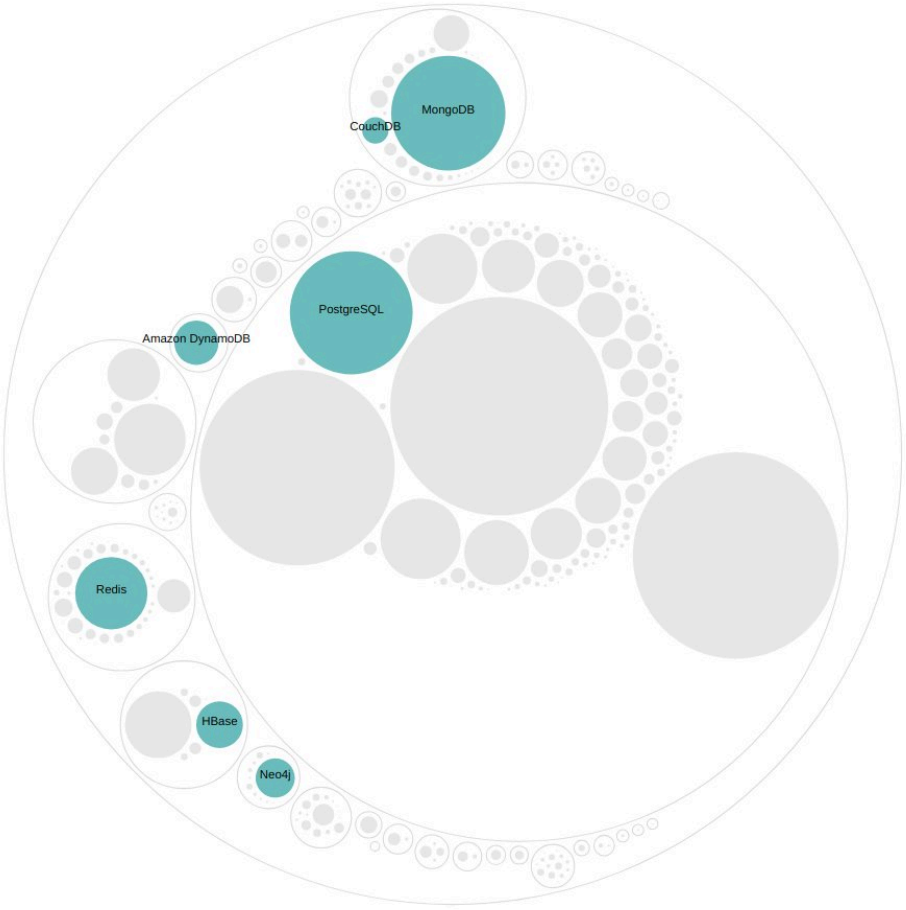
Amazon DynamoDB





1. https://docs.aws.amazon.com/en_us/dynamodb/ documentación oficial en inglés (HTML, PDF, Kindle, GitHub)
2. https://docs.aws.amazon.com/es_es/dynamodb/ documentación en español (HTML, PDF)

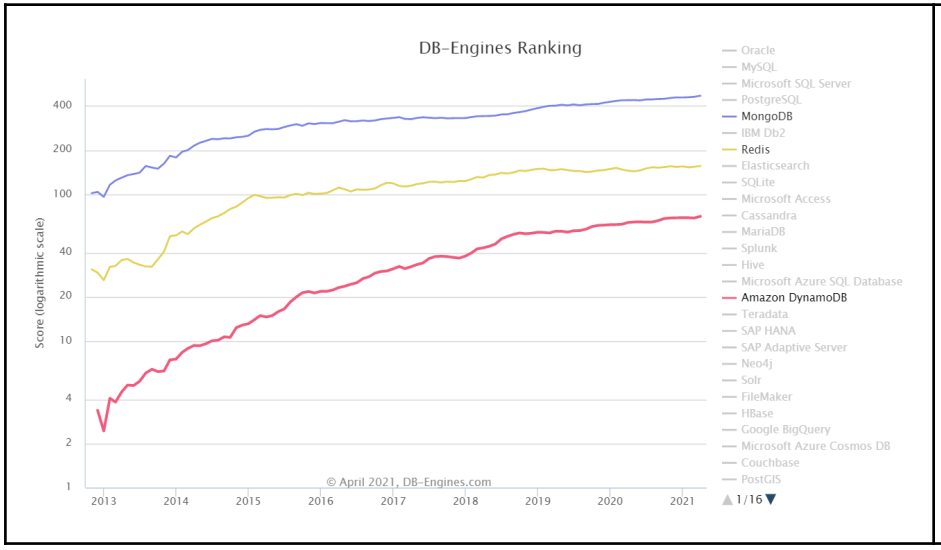
Cloud key/value store
data model: **key-value *plus***

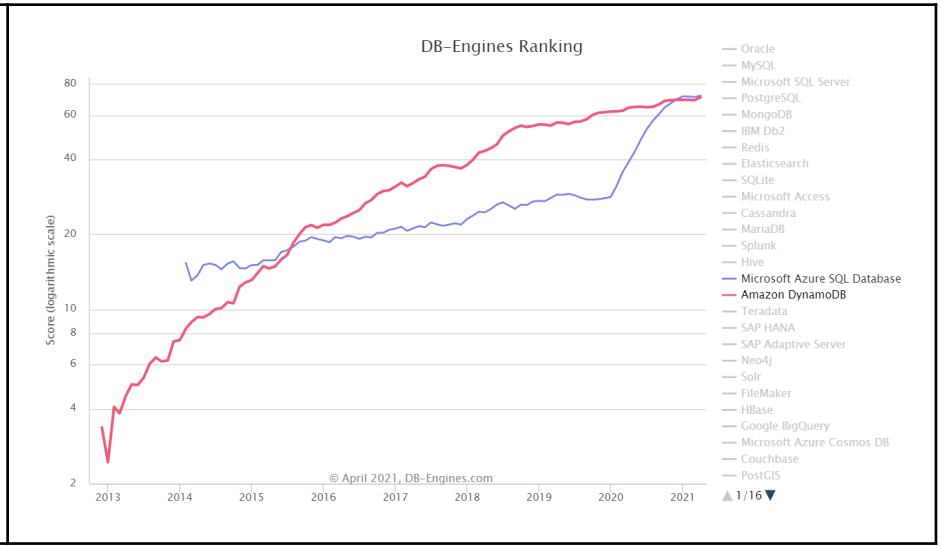
DB-Engines Ranking https://db-engines.com/en/ranking
Hosted, scalable database service by Amazon with the data stored in Amazon's cloud

#16 Overall
#2 Key-value stores
#2 Document stores

```
Como Document store
   1. MongoDB                Document          441.90
   2. Amazon DynamoDB        Multi-model        77.45

Como Key-value
   1. Redis                  Key-value         173.55
   2. Amazon DynamoDB        Multi-model        77.45
```





Amazon DynamoDB es un servicio de base de datos NoSQL totalmente administrado que ofrece un rendimiento rápido y predecible, así como una perfecta escalabilidad. Puede utilizar Amazon DynamoDB para crear una tabla de base de datos capaz de almacenar y recuperar cualquier cantidad de datos, así como de satisfacer cualquier nivel de tráfico de solicitud. Amazon DynamoDB distribuye automáticamente los datos y el tráfico de la tabla por una cantidad suficiente de servidores como para administrar la capacidad de solicitud especificada por el cliente y la cantidad de datos almacenados, al mismo tiempo que mantiene un desempeño uniforme y rápido.

Amazon DynamoDB *is a fully managed proprietary NoSQL database service that supports* ***key-value*** *and* ***document data structures*** *and is offered by Amazon.com as part of the Amazon Web Services portfolio.* ***DynamoDB*** *exposes a similar data model to and derives its name from* ***Dynamo****, but has a different underlying implementation.* ***DynamoDB*** *was announced by Amazon CTO Werner Vogels on January 18, 2012, and is presented as an evolution of Amazon* ***SimpleDB*** *solution.*

Initial release date: January 18, 2012.
License: **Proprietary**
Developed by: **Amazon.com**

https://en.wikipedia.org/wiki/Amazon_DynamoDB

You can store as many items as you want in any DynamoDB table.

Each ***item*** *(the equivalent of a* ***row*** *in an SQL database) can hold as many attributes as you want, although there is a hard size limit of 400 KB per item.*

While some folks may be reticent to trust a cloud database that's managed by someone else, a variety of forward-thinking tech companies, including ***Airbnb****,* ***Adobe****,* ***Siemens****, and* ***Comcast****, have taken the plunge and use DynamoDB as one of the core databases driving their platforms.*

https://aws.amazon.com/es/dynamodb/ (Casos practicos)

All data in DynamoDB is stored on high-performing Solid State Disks (SSDs) and ***automatically replicated*** *across multiple availability zones within an AWS region (which guarantees redundancy even within a single region).*

Dynamo paper (2007) → Dynamo: Amazon's Highly Available Key-value Store

https://www.dynamodbguide.com/the-dynamo-paper/
In 2004, Amazon.com was growing rapidly and was starting to hit the upper scaling limits of its Oracle database.
Key aspects of Dynamo

1. No Relational model
2. **Availability** > Consistency
3. Infinitely Scalable

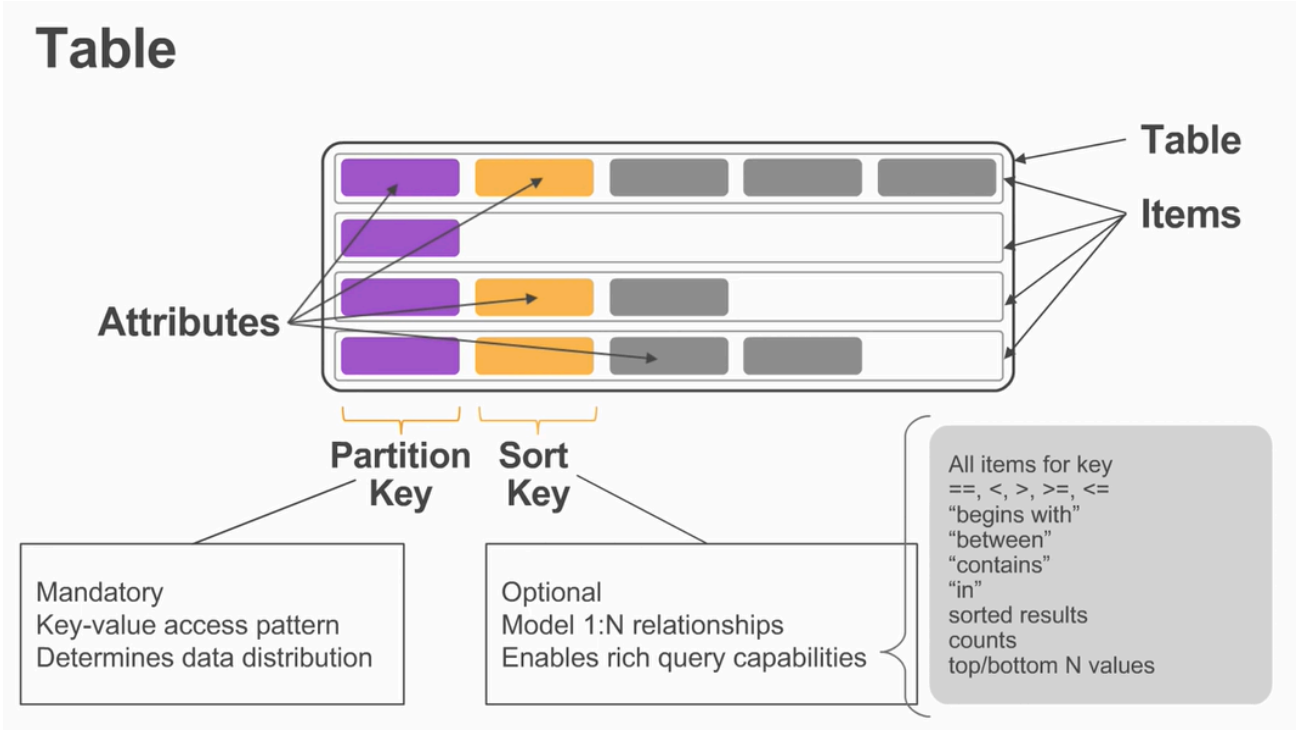
Is DynamoDB the same product described in Amazon's Dynamo paper?
https://www.quora.com/Is-DynamoDB-the-same-product-described-in-Amazons-Dynamo-paper

items in DymamoDB → *rows* in Postgresql.

Los *ítems*, tienen uno o más *atributos*, que se corresponden con las columnas en el modelo relacional.

Podemos acceder a los registros a través de un índice secundario, no sólo a través de la clave primaria.

También se pueden hacer consultas por rangos.



Video = [SQL to NoSQL Best Practices with Amazon DynamoDB - AWS July 2016 Webinar Series]
Slides = [https://www.slideshare.net/AmazonWebServices/sql-to-nosql-best-practices-with-amazon-dynamodb-aws-july-2016-webinar-series]

Another important thing to note about consistency is that DynamoDB *supports only item-level consistency*, which is analogous to row-level consistency in RDBMSs. There are no atomic operations across items, which means no consistency for batch operations. And when you run queries against indexes or whole tables, do not ever expect that the result set will be 100 percent up-to-date. Item-level consistency is a good thing to have, but if consistency across items is a necessity for your use case, you should explore other databases.

DynamoDB is not free! It is a paid service. AWS offers a free tier for virtually all of its services so that you can kick the tires without immediately incurring costs, but make sure to check the pricing guide before you start going through the examples in this book.

Si queremos correr DynamoDB
DynamoDB Local for Desktop Development
https://aws.amazon.com/es/blogs/aws/dynamodb-local-for-desktop-development/

```
$ cd ~/dynamodb_local
$ java -Djava.library.path=./DynamoDBLocal_lib -jar DynamoDBLocal.jar -sharedDb
Initializing DynamoDB Local with the following configuration:
Port: 8000
InMemory:   false
DbPath:     null
SharedDb:   true
shouldDelayTransientStatuses: false
CorsParams: *
```

Before you can access DynamoDB programmatically or through the AWS Command Line Interface (AWS CLI), you must configure your credentials to enable authorization for your applications. Downloadable DynamoDB requires any credentials to work, as shown in the following example.

```
$ aws configure
AWS Access Key ID: "fakeMyKeyId"
AWS Secret Access Key: "fakeSecretAccessKey"
```

```
C:\Users\aaizemberg> aws dynamodb list-tables --endpoint-url http://localhost:8000

You must specify a region. You can also configure your region by running "aws configure".

C:\Users\aaizemberg> aws configure
AWS Access Key ID [****************eyId]:
```

```
AWS Secret Access Key [****************sKey]:
Default region name [us-east-1]:
Default output format [text]: json

C:\Users\aaizemberg> aws dynamodb list-tables --endpoint-url http://localhost:8000
{
    "TableNames": []
}
```

Installing the AWS Command Line Interface
https://docs.aws.amazon.com/cli/latest/userguide/installing.html
https://docs.aws.amazon.com/cli/latest/userguide/getting-started-install.html

`$ aws dynamodb help` https://docs.aws.amazon.com/cli/latest/reference/dynamodb/index.html#cli-aws-dynamodb
Available Commands (sunburst)
nueva versión del sunburst

Listando las tablas

```
$ aws dynamodb list-tables --endpoint-url
http://localhost:8000
{
    "TableNames": [
        "Books", "MusicCollection", "ShoppingCart"
    ]
}
```

```
C:\Program Files\Amazon\AWSCLI\bin>aws dynamodb
list-tables --endpoint-url http://localhost:8000
{
    "TableNames": [
        "Books", "MusicCollection", "ShoppingCart"
    ]
}
```

create-table

```
$ aws dynamodb create-table --table-name ShoppingCart --attribute-definitions
AttributeName=ItemName,AttributeType=S --key-schema AttributeName=ItemName,KeyType=HASH --provisioned-throughput
ReadCapacityUnits=1,WriteCapacityUnits=1 --endpoint-url http://localhost:8000


$ aws dynamodb create-table --table-name MusicCollection --attribute-definitions
AttributeName=Artist,AttributeType=S AttributeName=SongTitle,AttributeType=S --key-schema
AttributeName=Artist,KeyType=HASH AttributeName=SongTitle,KeyType=RANGE --provisioned-throughput
ReadCapacityUnits=1,WriteCapacityUnits=1 --endpoint-url http://localhost:8000
```

RCU (ReadCapacityUnits) y WCU (WriteCapacityUnits)

```
$ aws dynamodb list-tables --endpoint-url http://localhost:8000

// Después de ejecutar el 'aws dynamodb create-table …' obtenemos un JSON con el resultado.

 {
  TableDescription:  {
    AttributeDefinitions: [ {
  AttributeName: "ItemName",
  AttributeType: "S"
}],
    CreationDateTime: 1475032237.808,
    ItemCount: 0,
    KeySchema: [ {
  AttributeName: "ItemName",
  KeyType: "HASH"
}],
    ProvisionedThroughput:  { ... },
    TableArn: "arn:aws:dynamodb:...:table/ShoppingCart",
```

```
    TableName: "ShoppingCart",
    TableSizeBytes: 0,
    TableStatus: "CREATING"
  }
}
```

Si hacemos un 'describe', es de esperar que el *TableStatus* cambie de estado. (CREATING → ACTIVE)

```
$ aws dynamodb describe-table --table-name ShoppingCart --endpoint-url http://localhost:8000
o
$ aws dynamodb describe-table --table-name MusicCollection --endpoint-url http://localhost:8000

...
    TableStatus: "ACTIVE"
...
```

Partition Key = Hash (// PK no es Primary Key)
Sort Key = Range

DynamoDB's Data Types

Type	Symbol	Description	JSON Example
String	S	A typical string like you'd find in most programming languages.	"S": "this is a string"
Number	N	Any integer or float. Sent as a string for the sake of compatibility between client libraries.	"N": "98", "N":"3.141592"
Binary	B	Base64-encoded binary data of any length (within item size limits).	"B": "4SrNYKrcv4wjJczEf6u+TgaT2YaWGgU76YPhF"
Boolean	BOOL	true or false.	"BOOL": false
Null	NULL	A null value. Useful for missing values.	"NULL": true

Type	Symbol	Description	JSON Example
String set	SS	A set of strings.	"SS": ["Larry", "Moe"]
Number set	NS	A set of numbers.	"NS": ["42", "137"]
Binary set	BS	A set of binary strings.	"BS": ["TGFycnkK", "TW9lCg=="]
List	L	A list that can consist of data of any scalar type, like a JSON array. You can mix scalar types as well.	"L": [{"S": "totchos"}, {"N": "741"}]
Map	M	A key-value structure with strings as keys and values of any type, including sets, lists, and maps.	"M": {"FavoriteBook": {"S": "Old Yeller"}}

Warning! No Empty Strings

JSON *en DynamoDB*

límites: Un ítem no puede superar los **400 KB**

Aquí hay un ejemplo de operación put que usa un JSON:

```
$ aws dynamodb put-item --table-name Books \
--item '{
  "ISBN": { "N": "98765" },
  "PublisherInfo": {"S": "Something" },
  "PublishYear": { "N": "2012" },
  "Title": { "S": "Moby Dick" }
}'
```

JSON	DynamoDB JSON

{
  "url": "https://aws.amazon.com/es/dynamodb/"
}
	{
	  "url": {
	  "S": "https://aws.amazon.com/es/dynamodb/"
	  }
	}

Basic Read/Write Operations

```
$ aws dynamodb put-item --table-name ShoppingCart --item '{"ItemName": {"S": "Tickle Me Elmo"}}' --endpoint-url
http://localhost:8000

// Si les da error, probar de esta manera:
$ aws dynamodb put-item --table-name ShoppingCart --item file://item1.json --endpoint-url http://localhost:8000
donde 'item1.json' → {"ItemName": {"S": "Tickle Me Elmo"}}

$ aws dynamodb put-item --table-name ShoppingCart --item '{"ItemName": {"S": "1975 Buick LeSabre"}}'
--endpoint-url http://localhost:8000

$ aws dynamodb put-item --table-name ShoppingCart --item '{"ItemName": {"S": "Ken Burns: the Complete Box Set"}}'
--endpoint-url http://localhost:8000
```

Si queremos ver todos los registros insertados, similar al SQL: SELECT * FROM ShoppingCart

```
$ aws dynamodb scan --table-name ShoppingCart --endpoint-url http://localhost:8000
{
  "Count": 3,
```

```
  "Items": [
  {"ItemName": { "S": "1975 Buick LeSabre" } },
  {"ItemName": { "S": "Ken Burns: the Complete Box Set" }},
  {"ItemName": { "S": "Tickle Me Elmo" }}
  ],
  "ScannedCount": 3,
  "ConsumedCapacity": null
}
```

Contando la cantidad de registros de una tabla (Select COUNT(*) from <table>)

```
$ aws dynamodb scan --table-name ShoppingCart --select "COUNT" --endpoint-url http://localhost:8000
{
    "Count": 3,
    "ScannedCount": 3,
    "ConsumedCapacity": null
}
```

Buscando un ítem en particular.

```
$ aws dynamodb get-item --table-name ShoppingCart --key '{"ItemName": {"S": "Tickle Me Elmo"}}' --endpoint-url
http://localhost:8000
{
  "Item": {
    "ItemName": { "S": "Tickle Me Elmo"}
  }
}
```

si no funciona el comando anterior, usar este:

```
$ aws dynamodb get-item --table-name ShoppingCart --key file://item1.json --endpoint-url http://localhost:8000
```

Si queremos borrar un item

```
$ aws dynamodb delete-item --table-name ShoppingCart --key '{"ItemName": {"S": "Tickle Me Elmo"}}' --endpoint-url
http://localhost:8000
```
o
```
$ aws dynamodb delete-item --table-name ShoppingCart --key file://item1.json --endpoint-url http://localhost:8000
```

Two Key Types, Many Possibilities
DynamoDB es en esencia un key-value store

- **hash key** (la clave de acceso que también funciona como la clave para hacer las particiones. ***partition key***)
- **range key** (la clave que que se usa para hacer búsquedas ordenadas)

Imagine a table storing information about **books** in which each book item has **two properties**: a **title** (string) and the **year published** (number).

```
$ aws dynamodb create-table --table-name Books --attribute-definitions AttributeName=Title,AttributeType=S
AttributeName=PublishYear,AttributeType=N --key-schema AttributeName=Title,KeyType=HASH
AttributeName=PublishYear,KeyType=RANGE --provisioned-throughput ReadCapacityUnits=1,WriteCapacityUnits=1
--endpoint-url http://localhost:8000
```

En este caso, el título del libro se va a usar como clave de tipo HASH y la fecha de publicación como RANGE

Vamos a crear 3 registros. Supongamos que el libro 'Moby Dick' tiene 3 ediciones.

```
$ aws dynamodb put-item --table-name Books \
--item '{
  "Title": {"S": "Moby Dick"},
  "PublishYear": {"N": "1851"},
  "ISBN": {"N": "12345"} }' --endpoint-url http://localhost:8000
```

```
$ aws dynamodb put-item --table-name Books --item file://book1.json --endpoint-url http://localhost:8000
donde 'book1.json' → { "Title": {"S": "Moby Dick"}, "PublishYear": {"N": "1851"},  "ISBN": {"N": "12345"} }

$ aws dynamodb put-item --table-name Books \
--item '{
  "Title": {"S": "Moby Dick"},
  "PublishYear": {"N": "1971"},
  "ISBN": {"N": "23456"},
  "Note": {"S": "Out of print"}
}' --endpoint-url http://localhost:8000

$ aws dynamodb put-item --table-name Books \
--item '{
  "Title": {"S": "Moby Dick"},
  "PublishYear": {"N": "2008"},
  "ISBN": {"N": "34567"}
}' --endpoint-url http://localhost:8000
```

Si queremos buscar los libros con título "Moby Dick" que fueron publicados después de 1980.

Consulta:

```
$ aws dynamodb query --table-name Books \
  --expression-attribute-values '{ ":title": {"S": "Moby Dick"}, ":year": {"N": "1980"} }' \
  --key-condition-expression 'Title = :title AND PublishYear > :year' --endpoint-url http://localhost:8000
```

Resultado:

```
{
  "Count": 1,
  "Items": [{ "PublishYear": { "N": "2008" }, "ISBN": { "N": "34567" }, "Title": { "S": "Moby Dick" } } ],
```

```
  "ScannedCount": 1,
  "ConsumedCapacity": null
}
```

Si queremos proyectar solo el ISBN en la respuesta?

```
$ aws dynamodb query --table-name Books \
  --expression-attribute-values '{":title": {"S": "Moby Dick"},":year": {"N": "1900"}}' \
  --key-condition-expression 'Title = :title AND PublishYear > :year' \
  --projection-expression 'ISBN' \
  --endpoint-url http://localhost:8000

{
  "Count": 2,
  "Items": [ { "ISBN": { "N": "23456" } }, { "ISBN": { "N": "34567" } } ],
  "ScannedCount": 2,
  "ConsumedCapacity": null
}
```

¿Qué pasaría si proyecto un atributo que no está presente en todos los registros?

```
$ aws dynamodb query --table-name Books \
  --expression-attribute-values '{":title": {"S": "Moby Dick"},":year": {"N": "1900"}}' \
  --key-condition-expression 'Title = :title AND PublishYear > :year' \
  --projection-expression 'Note' \
  --endpoint-url http://localhost:8000

{
  "Count": 2,
  "Items": [
```

```
    { "Note": { "S": "Out of print" } },
    {}                                          # → el 2do registro, no tiene el atributo 'Note'
  ],
  "ScannedCount": 2,
  "ConsumedCapacity": null
}
```

Momento de reflexión.

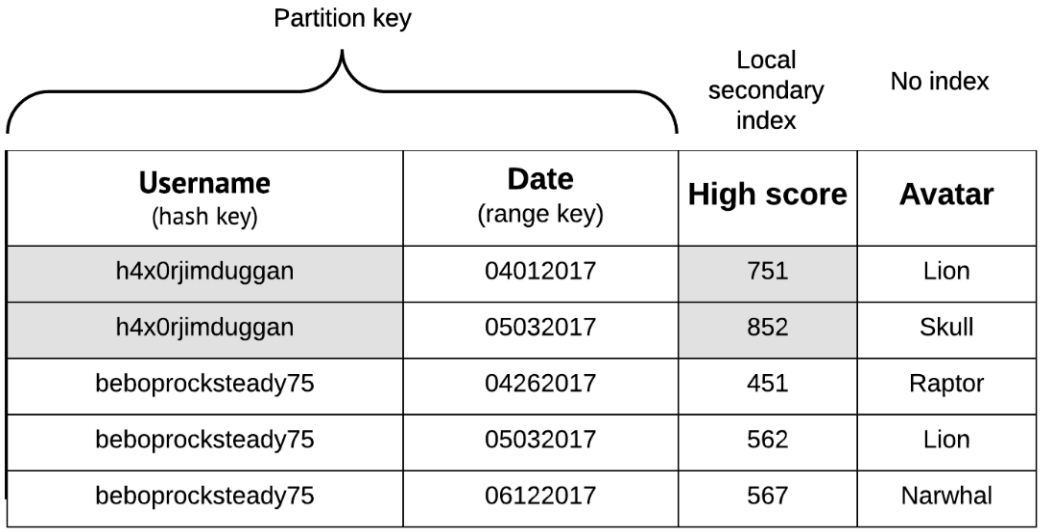
1. Cuando creamos la tabla, solo habíamos mencionado 2 atributos, el título y el año de publicación.
2. Cuando insertamos registros, agregamos en el ítem (o documento) otros atributos (ISBN y Nota).
    a. En el DB-ranking dice que el **data scheme** es **schema-free**, Uds que piensan?
3. El JSON no es como el que veníamos usando en MongoDB o CouchDB. (**data typing**: yes o no)

Indices

En DynamoDB, más allá del *partition key* (*hash key* y el *range key*) los índices secundarios pueden ser de 2 tipos: ***local*** (LSI*)* o ***global*** (GSI*).*

Local Secondary Indexes

Local secondary indexes (LSIs) let you query or scan attributes outside of your primary hash and sort key.




```
$ aws dynamodb query --table-name BonestormData \
  --expression-attribute-values '{":user": {"S": "h4x0rjimduggan"},":score": {"N": "800"}}' \
  --key-condition-expression 'Username = :user AND HighScore > :score' \
  --projection-expression 'Date'
```

Always bear in mind that LSIs can't be modified after a table has been created, so be extra careful to plan for them in advance

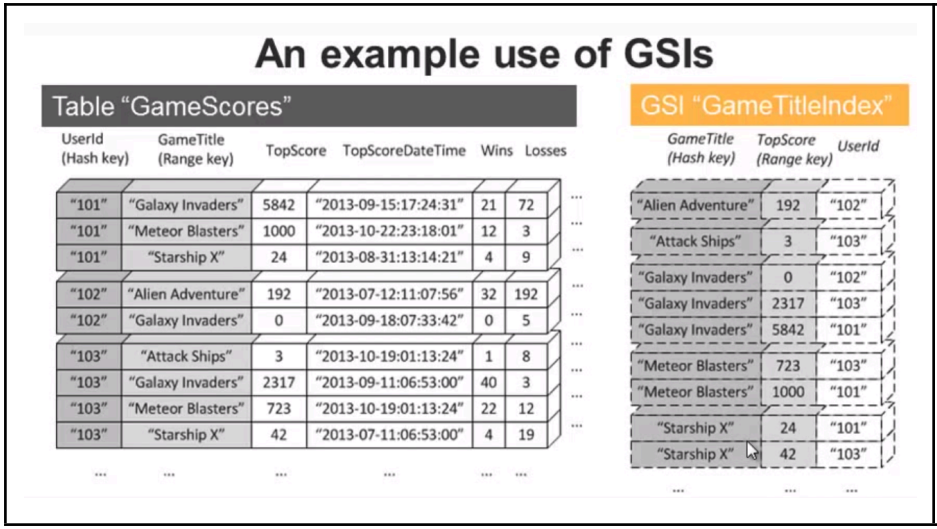
Global Secondary Indexes

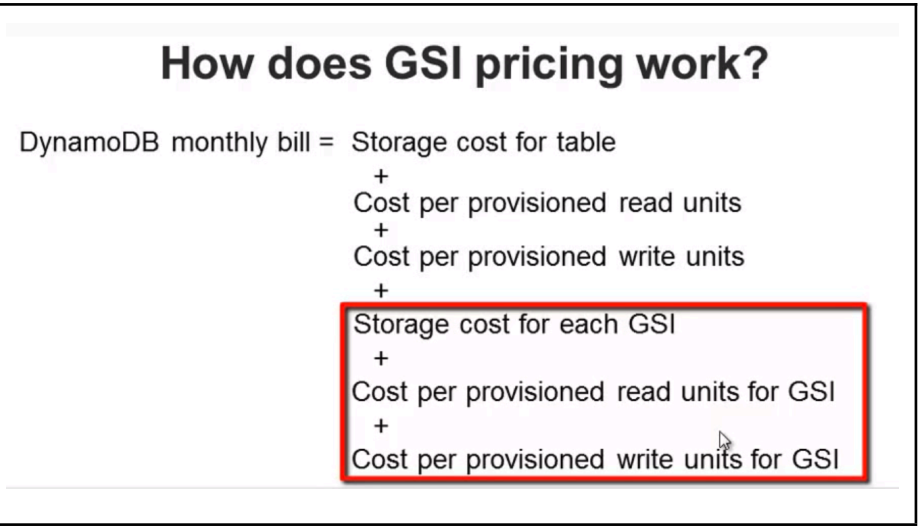
Global secondary indexes (GSIs) are indexes that aren't restricted to items with the same partition key. The "global" here essentially means "any attribute in the table."

Una cosa importante a tener en cuenta es que los GSI se pueden modificar después de que se haya creado una tabla, mientras que los LSI no.

```
$ aws dynamodb query --table-name BonestormData \
  --expression-attribute-values '{":avatar": {"S": "Lion"}}' \
  --key-condition-expression 'Avatar = :avatar' \
  --projection-expression 'Username, Date'
```

SQL	DynamoDB
Tables	Tables
Rows	Items
Columns	Attributes
Primary Key (can have several columns)	Primary Key (Partition key and an optional Sort key)
Indexes	Local Secondary Indexes (Same partition key but a different sort key)
Views	Global Secondary Indexes (Different partition key and a sort key)





Fin dia 1

Dia 2

Building a Streaming Data Pipeline

```
$ aws dynamodb create-table --cli-input-json file://sensor-data-table.json
```

file: **dynamodb/sensor-data-table.json**

```
{
  "TableName": "SensorData",
  "KeySchema": [
    {
      "AttributeName": "SensorId",
      "KeyType": "HASH"
    },
```

```
    {
      "AttributeName": "CurrentTime",
      "KeyType": "RANGE"
    }
  ],
  "AttributeDefinitions": [
    {
      "AttributeName": "SensorId",
      "AttributeType": "S"
    },
    {
      "AttributeName": "CurrentTime",
      "AttributeType": "N"
    },
    {
      "AttributeName": "Temperature",
      "AttributeType": "N"
    }
  ],
  "LocalSecondaryIndexes": [
    {
      "IndexName": "TemperatureIndex",
      "KeySchema": [
        {
          "AttributeName": "SensorId",
          "KeyType": "HASH"
        },
        {
          "AttributeName": "Temperature",
          "KeyType": "RANGE"
        }
      ],
      "Projection": {
        "ProjectionType": "ALL"
      }
    }
```

```
  ],
  "ProvisionedThroughput": {
    "ReadCapacityUnits": 2,
    "WriteCapacityUnits": 2
  }
}
```

SensorData		
KeySchema		
SensorId	**CurrentTime**	**Temperature**
HASH	RANGE	
S	N	N
LocalSecondaryIndexes		LocalSecondaryIndexes

Throughput

1 = 4 para las Lecturas
1 = 1 para las Escrituras

```
$ aws dynamodb create-table  \
  --table-name ShoppingCart  \
  --attribute-definitions AttributeName=ItemName,AttributeType=S \
  --key-schema AttributeName=ItemName,KeyType=HASH \
  --provisioned-throughput ReadCapacityUnits=1,WriteCapacityUnits=1
```

Las unidades de capacidad de lectura y escritura se manejan de manera diferente en DynamoDB.
Las unidades de capacidad de lectura (RCU) se miden en unidades de **4** KB por segundo.
Las unidades de capacidad de escritura (WCU) se miden en unidades de **1** KB por segundo.

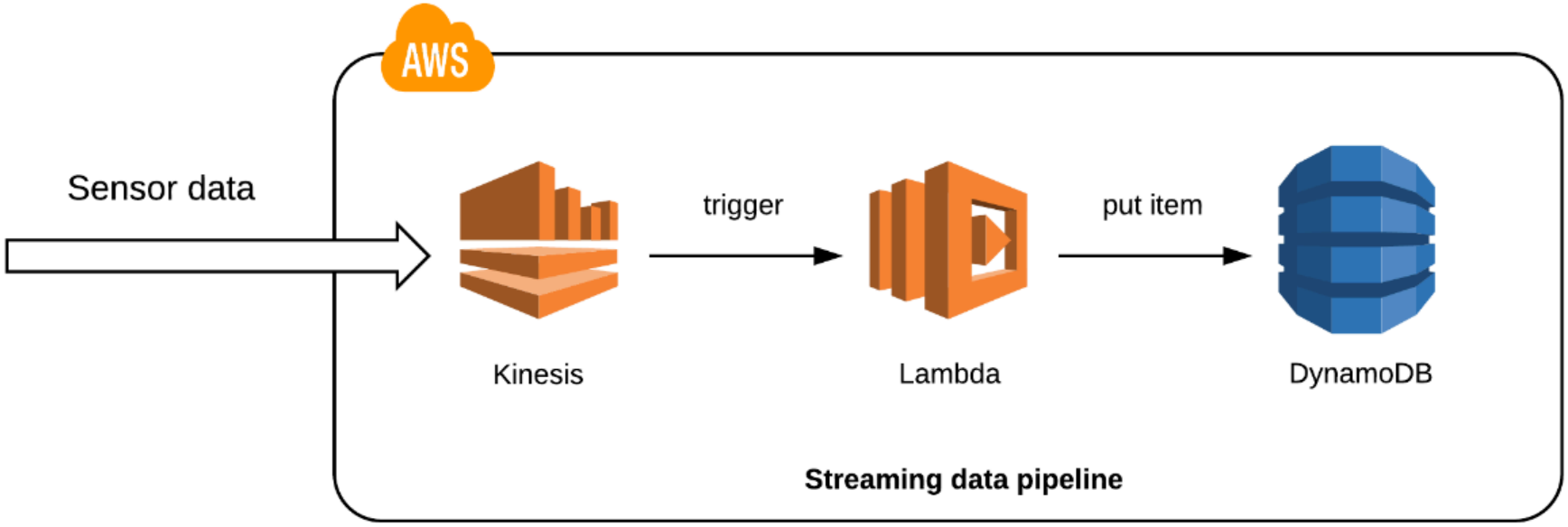
Strongly consistent reads *are about* ***twice as expensive*** *as* ***eventually consistent*** *reads.*

https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.ProvisionedThroughput.html

La configuración de rendimiento de una tabla no está fija en todo momento y se puede actualizar a voluntad.
Podemos empezar con RCU=1, WCU=1 e ir modificando estos valores si notamos que necesitamos una mejor performance.

Data Streams vs Procesamiento Batch

A Real Time, Streaming, Functional Data Pipeline



file: **dynamodb/ProcessKinesisRecords.js**

```
var AWS = require('aws-sdk');
var DynamoDB = new AWS.DynamoDB({
  apiVersion: '2012-08-10',
  region: 'us-east-1',
});

exports.kinesisHandler = function(event, context, callback) {
  // We only need to handle one record at a time
  var kinesisRecord = event.Records[0];
```

```
// The data payload is base 64 encoded and needs to be decoded to a string
var data   =
  Buffer.from(kinesisRecord.kinesis.data, 'base64').toString('ascii');
// Create a JSON object out of that string
var obj    = JSON.parse(data);
var sensorId    = obj.sensor_id,
    currentTime = obj.current_time,
    temperature = obj.temperature;

// Define the item to write to DynamoDB
var item = {
  TableName: "SensorData",
  Item: {
    SensorId: { S: sensorId },
    CurrentTime: { N: currentTime.toString() },
    Temperature: { N: temperature.toString() }
  }
};

// Perform a put operation, logging both successes and failures
DynamoDB.putItem(item, function(err, data) {
  if (err) {
    console.log(err, err.stack);
    callback(err.stack);
  } else {
    console.log(data);
    callback(null, data);
  }
});
```

```
}

... muchos pasos de configuración de otros productos (!=DynamoDB), Kinesis, lambda, y detalles de AWS.

$ export DATA=$( echo '{ "sensor_id":"sensor-1", "temperature":99.9,"current_time":123456789 }' | base64)
$ export STREAM_NAME=temperature-sensor-data
$ aws kinesis put-record \
  --stream-name ${STREAM_NAME} \
  --partition-key sensor-data \
  --data ${DATA}

$ aws dynamodb scan --table-name SensorData
{
  "Count": 1,
  "Items": [
    { "CurrentTime": { "N": "123456789" },
      "Temperature": { "N": "99.9" },
      "SensorId": { "S": "sensor-1" }
    }
  ]
}
```

Tarea

Find one or more DynamoDB client libraries for your favorite programming language.
Explore how to perform CRUD operations using that library.

Supported programming languages: .Net, ColdFusion, Erlang, Groovy, Java, JavaScript, Perl, PHP, Python, Ruby.
Java → Getting Started with Java and DynamoDB - Amazon DynamoDB
Python → Getting Started Developing with Python and DynamoDB - Amazon DynamoDB

boto3 Configuration (cliente python)
Quickstart — Boto3 documentation

boto3 & **DynamoDB**
Boto3 documentation

You can create the credential file yourself. By default, its location is at **~/.aws/credentials**:

[default]
aws_access_key_id = YOUR_ACCESS_KEY
aws_secret_access_key = YOUR_SECRET_KEY

You may also want to set a default region. This can be done in the configuration file. By default, its location is at **~/.aws/config**:

[default]
region = us-east-1
output = json

Alternatively, you can pass a region_name when creating clients and resources.

Python

1. Create a table
2. Load sample data
3. Create, Read, Update, and Delete an Item
4. Query and Scan the Data
5. (Optional) Delete the Table

Fin del dia 2
On Day 3, we'll begin pumping data into Kinesis and letting Lambda do its magic.

Dia 3

Building an “Internet of Things” System Around DynamoDB

Mock Data Inputs
dynamodb/upload-sensor-data.rb
https://media.pragprog.com/titles/pwrdata/code/dynamodb/upload-sensor-data.rb

Para poder correr el script, necesitamos instalar algunas gemas de ruby.

```
$ gem install random-walk aws-sdk
```

1 sensor, 1000 muestras.

```
$ ruby upload-sensor-data.rb temp-sensor-1 1000
```

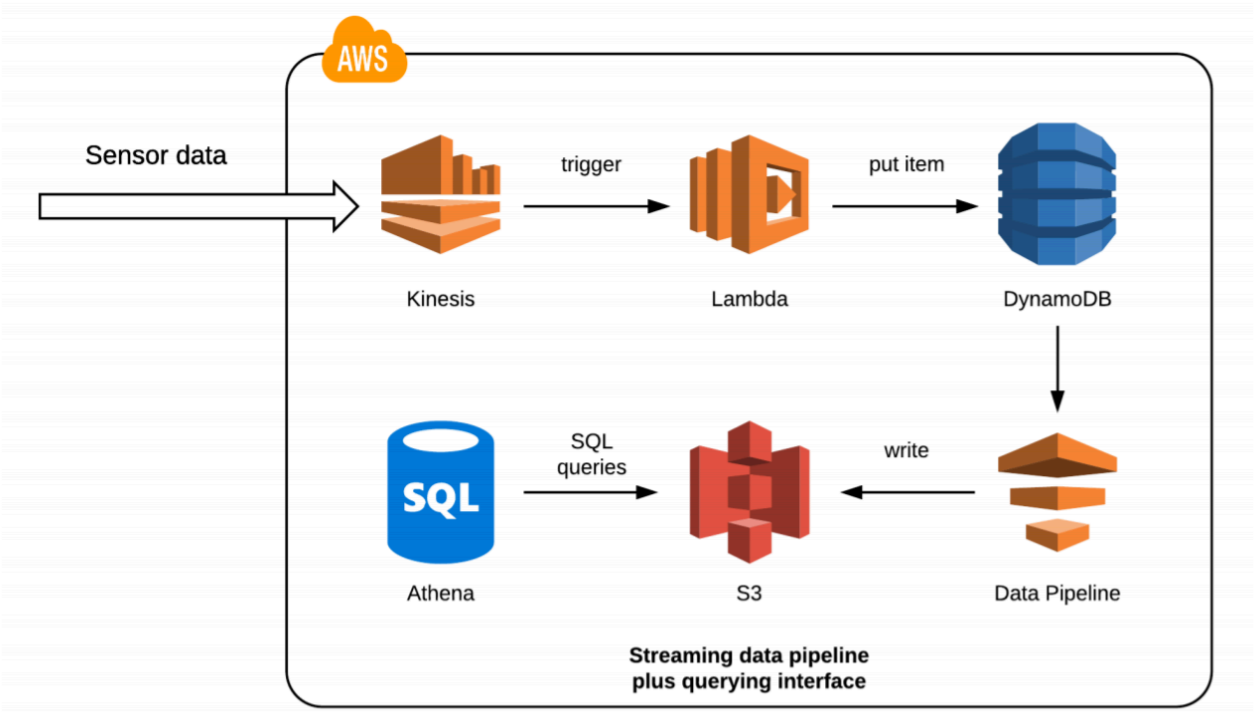
10 sensores, 1000 muestras.

```
$ for n in {1..10}; do
    ruby upload-sensor-data.rb sensor-${n} 1000 &
  done
```

Para ver cuántos registros han ingresado.

```
$ aws dynamodb scan --table-name SensorData --query Count
```

An **SQL** Querying Interface (...dynamoDB → S3 ← Athena)



DynamoDB → S3 (Simple Storage Service)

```
CREATE EXTERNAL TABLE sensor_data (
  sensorid struct<s:string>,
  currenttime struct<n:bigint>,
  temperature struct<n:float>
)
  ROW FORMAT SERDE 'org.openx.data.jsonserde.JsonSerDe'
  with serdeproperties ('paths'='SensorId,CurrentTime,Temperature')
  LOCATION 's3://YOUR-S3-BUCKET/';
```

Amazon Athena - https://aws.amazon.com/athena/
Start querying data instantly. Get results in seconds. Pay only for the queries you run.

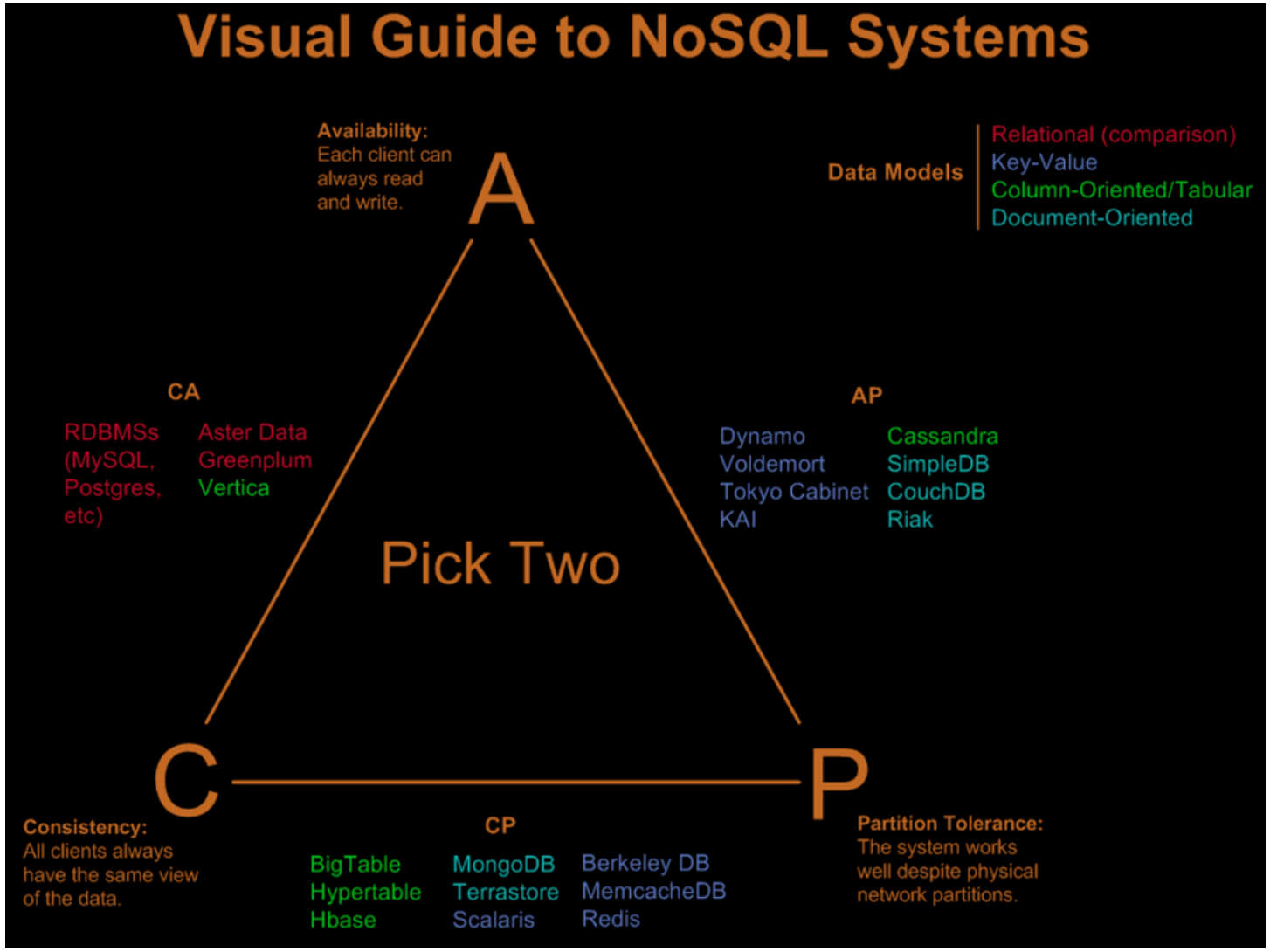
```
/* List the sensors for which you have data */
SELECT DISTINCT sensorid.s AS SensorId FROM sensor_data;
/* Get the number of readings from a specific sensor */
SELECT COUNT(*) AS NumberOfReadings FROM sensor_data
  WHERE sensorid.s = 'some-sensor-id';
/* Find the average temperature measured by a specific sensor */
SELECT AVG(temperature.n) AS AverageTemp FROM sensor_data
  WHERE sensorid.s = 'some-sensor-id';
/* Find the average temperature across all sensors */
SELECT AVG(temperature.n) AS AverageTempAllSensors FROM sensor_data;
/* Find the average temperature from a specific sensor */
SELECT AVG(temperature.n) AS AverageTemp FROM sensor_data
  WHERE sensorid.s = 'some-sensor-id';
/* Find the maximum temperature across all sensors */
SELECT MAX(temperature.n) AS MaxTemp FROM sensor_data;
/* Find the standard deviation for temperature across all sensors */
SELECT STDDEV(temperature.n) AS StdDev FROM sensor_data;
```

---

sensor-data → sensor-data-YYYY-MM-DD

Una sugerencia que hacen en el libro, es dividir la tabla, para que cada día pueda estar alojado en un bucket diferente.

Fin dia 3.


Visual Guide to NoSQL Systems
Availability:
Each client can always read and write.
A
Data Models
Relational (comparison)
Key-Value
Column-Oriented/Tabular
Document-Oriented
CA
RDBMSs (MySQL, Postgres, etc)
Aster Data
Greenplum
Vertica
AP
Dynamo
Voldemort
Tokyo Cabinet
KAI
Cassandra
SimpleDB
CouchDB
Riak
Pick Two
C
P
Consistency:
All clients always have the same view of the data.
CP
BigTable
Hypertable
Hbase
MongoDB
Terrastore
Scalaris
Berkeley DB
MemcacheDB
Redis
Partition Tolerance:
The system works well despite physical network partitions.

DynamoDB's Strengths

The strengths of DynamoDB have been on display throughout this chapter. It enables you to skip all the installation and setup steps and get started building immediately. And if you build something small, there are few intrinsic limits to how big it can get (if, of course, you have the budget for it). DynamoDB can be tricky in places, especially when it comes to things like capacity provisioning, planning for indexes, and coming up with a data model that enables you to take advantage of DynamoDB's performance and scalability. But getting past the tricky parts unlocks a lot of possibilities.

Although DynamoDB requires you to give up some of the querying power of relational databases, its data and querying models provide some powerful constructs, such as indexes and range queries, that you won't find in a lot of NoSQL databases. And if you run into barriers, such as the inability to write SQL-style queries for tables, you may be able to use an external service to fill in the gaps, as we did on Day 3. Beyond all of this, DynamoDB's feature set continues to expand, as does the constellation of services surrounding DynamoDB in the AWS ecosystem.

Fortalezas de DynamoDB

Las fortalezas de DynamoDB se han exhibido a lo largo de este capítulo. Le permite omitir todos los pasos de instalación y configuración y comenzar a construir de inmediato. Y si construyes algo pequeño, hay pocos límites intrínsecos a lo grande que puede llegar a ser (si, por supuesto, tienes el presupuesto para ello). DynamoDB puede ser complicado en algunos lugares, especialmente cuando se trata de cosas como el aprovisionamiento de capacidad, la planificación de índices y la creación de un modelo de datos que le permita aprovechar el rendimiento y la escalabilidad de DynamoDB. Pero superar las partes difíciles abre muchas posibilidades.

Aunque DynamoDB requiere que renuncies al poder de consulta de las bases de datos relacionales, sus datos y modelos de consulta proporcionan algunas construcciones poderosas, como índices y consultas de rango, que no encontrarás en muchas bases de datos NoSQL. Y si se encuentra con barreras, como la incapacidad de escribir consultas de estilo SQL para tablas, puede usar un servicio externo para llenar los vacíos, como lo hicimos el día 3. Más allá de todo esto, el conjunto de características de DynamoDB continúa expandiéndose, al igual que la constelación de servicios que rodean a DynamoDB en el ecosistema de AWS.

DynamoDB's Weaknesses

The drawbacks presented by DynamoDB are directly reminiscent of other NoSQL databases. Despite some interesting querying and modeling constructs, you're probably better off using Postgres or another relational database unless you know for sure that you're

Debilidades de DynamoDB

Los inconvenientes presentados por DynamoDB recuerdan directamente a otras bases de datos NoSQL. A pesar de algunas interesantes construcciones de consulta y modelado, probablemente sea mejor usar Postgres u otra base de datos relacional a menos

dealing with a DynamoDB-shaped problem (or maybe an HBase-shaped problem, or Mongo, and so on). Even if you have a use case that truly calls for a big NoSQL database, you may have trouble getting your data model to mesh well with DynamoDB's partitioning system. And as always, there's the cost. Any database you use is going to cost money somehow, but DynamoDB and others don't always make financial sense. There may be times when it makes sense to run databases yourself, maybe on your own hardware and maybe in the cloud. *Caveat emptor*.	que sepa con certeza que está lidiando con un problema en forma de DynamoDB (o tal vez un problema en forma de HBase, o Mongo, y así en). Incluso si tiene un caso de uso que realmente requiera una gran base de datos NoSQL, es posible que tenga problemas para que su modelo de datos se adapte bien al sistema de particionamiento de DynamoDB. Y como siempre, está el costo. Cualquier base de datos que se use va a costar dinero de alguna manera, pero DynamoDB y otros no siempre tienen sentido financiero. Puede haber ocasiones en que tenga sentido ejecutar bases de datos usted mismo, tal vez en su propio hardware y tal vez en la nube. *El comprador asume el riesgo*.
## Parting Thoughts Cloud databases aren't everyone's cup of tea. They require a lot of trust—trust in massive systems whose internals you can't see and trust in the future economic prospects of cloud providers themselves. These and other drawbacks are real, but there are some major upsides, as we hope to have shown in this chapter. We urge anybody with a strong interest in databases to give them a try. Even if you don't ever have a use case that requires a database like DynamoDB, we think that they're greatly empowering for developers and just downright fun. They turn your humble little laptop into a very powerful orchestration platform. DynamoDB is a good place to start but we focused on it here mostly due to its influence and market share. There are many other options. If you want a DynamoDB-style experience with a SQL favor, have a look at Google's BigQuery or Amazon's Redshift. For globally distributed databases, try Google Cloud Spanner, CockroachDB, or FaunaDB. We reckon that the database landscape is starting to lean strongly in the direction of the cloud and the ops-free lifestyle. The next edition of this book may even feature several cloud databases.	## Pensamientos finales Las bases de datos en la nube no son la taza de té para todos. Requieren mucha confianza: confianza en sistemas masivos cuyas partes internas no se pueden ver y confían en las perspectivas económicas futuras de los propios proveedores de la nube. Estos y otros inconvenientes son reales, pero hay algunas ventajas importantes, como esperamos haber mostrado en este capítulo. Instamos a cualquiera que tenga un gran interés en las bases de datos a probarlo. Incluso si nunca tiene un caso de uso que requiera una base de datos como DynamoDB, creemos que son muy motivadores para los desarrolladores y simplemente divertidos. Convierten su humilde laptop en una plataforma de orquestación muy poderosa. DynamoDB es un buen lugar para comenzar, pero nos centramos aquí principalmente debido a su influencia y participación en el mercado. Hay muchas otras opciones. Si desea una experiencia de estilo DynamoDB con un favor de SQL, eche un vistazo a BigQuery de Google o Redshift de Amazon. Para bases de datos distribuidas globalmente, pruebe Google Cloud Spanner, CockroachDB o

	FaunaDB. Consideramos que el panorama de la base de datos está comenzando a inclinarse fuertemente hacia la nube y el estilo de vida libre de operaciones. La próxima edición de este libro puede incluso presentar varias bases de datos en la nube.

Bonus track:

- Tables, Items, and Attributes - Amazon DynamoDB Core Concepts | Amazon Web Services

En la última charla que tuvimos con la gente de Etermax, nos comentaron sobre este tema: **Single-Table Design with DynamoDB**

- https://www.alexdebrie.com/posts/dynamodb-single-table/
- The DynamoDB Book by Alex DeBrie, Cap. 8, pag. 144

Mas info: https://aprendiendoaws.com/

- Amazon DynamoDB :: AWS Entrenamientos

Link directo a los 4 videos (via **Kevin Cortés Rodríguez** ex-alumno de Ing. Informática del ITBA)

- https://aprendiendoaws.com/media/videos/serverless-dynamodb-part01.mp4
- https://aprendiendoaws.com/media/videos/serverless-dynamodb-part02.mp4
- https://aprendiendoaws.com/media/videos/serverless-dynamodb-part03.mp4
- https://aprendiendoaws.com/media/videos/serverless-dynamodb-part04.mp4